

Utskrift till skärm

Utskrifter till skärm sköts av funktionen `printf()`. Den definieras på följande sätt:

```
#include <stdio.h>

int printf(const char * format, ...);
```

Denna funktion är en mycket mångsidig funktion, och kan hantera väldigt många olika former av utskrifter. Ser man på funktionsdefinitionen lite noggrannare ser man att den andra parametern är `...`. I detta sammanhang innebär `...` att där kan finnas ett fritt antal extra parametrar, allt från inga till väldigt många. Första parametern är en C-sträng som anger *hur* utskriften skall formateras, d.v.s. fältbredder, datatyper o.s.v., medan eventuella variabler och konstanter definieras separat istället för `...`. Ett exempel på hur programmet `Hello` kan skrivas i C:

```
#include <stdio.h>

int main () {
    printf ("Hello world!\n");
}
```

I detta fall så använder vi endast den första parametern för `printf()`, och fyller där i den text vi vill skriva ut. Inga extra variabler eller dylikt skall skrivas ut, så inga fler parametrar behövs. Notera att vi inkluderar endast `<stdio.h>`, och inte alls `<iostream>` som vi tidigare använt i så gott som alla program.

Utskrift av variabler

För att skriva ut variabler till skärmen måste vi använda extra parametrar till `printf()`. Vi måste i så fall även specificera i formatsträngen *var* och *hur* vi vill att variablerna skall skrivas ut. Här används vissa platshållare inne i formateringssträngen. Dessa reserverar plats för egentliga variabler som skrivs ut istället för platshållaren. Olika typer av platshållare används för olika datatyper, men alla har ett procenttecken (%) som första tecken för att indikera att det är ett platshållare. De vanligaste typerna är:

- `%d` för att skriva ut ett heltal (`int`).
- `%c` för att skriva ut ett tecken (`char`).
- `%s` för att skriva ut en C-sträng.
- `%f` för att skriva ut ett flyttal (`float` eller `double`).
- `%%` för att skriva ut ett normal procenttecken.

Ett exempel på en utskrift där man skriver ut ett heltal följt av ett flyttal kan se ut på följande sätt:

```
#include <stdio.h>
```

```
int main () {
    int Antal = 10;
    float Medeltal = 13.23;
    printf ("Antalet tal: %d, medeltalet är %f\n", Antal, Medeltal);
}
```

Vi har här skrivit in strängarna `%d` och `%f` på de ställen där vi vill att ett heltal respektive ett flyttal skall skrivas ut. Annan text kan placeras hur som helst runt dessa platshållare. Eftersom vi i formateringssträngen specificerar att vi skall skriva ut två tal måste dessa även ges till `printf()`, och det görs genom att ge två extra parametrar, d.v.s. `Antal` och `Medeltal`. Kör vi programmet ovan får vi:

```
% ./COutput2
Antalet tal: 10, medeltalet är 13.230000
%
```

Genom att placera in olika platshållare för variabler och konstanter i formateringssträngen kan vi uppnå mycket komplexa utskrifter. För att t.ex. skriva ut några C-strängar kan man göra på följande sätt:

```
#include <stdio.h>

int main () {
    char * S1 = "Här är";
    char * S2 = "en sträng i";
    char * S3 = "delar";

    printf ("%s %s %d %s.\n", S1, S2, 3, S3);
}
```

Har har vi ingen egentlig "ren" text i formateringssträngen om man borträknar mellanslagen, punkten och radbytet. Istället finns där fyra platshållare för tre strängar och ett heltal. Här används `%s` för att representera en C-sträng. Notera att `%s` inte kan användas för att direkt skriva ut variabler av typen `string`, utan man måste i så fall använda metoden `c_str()`. Kör man programmet ovan får man följande utskrift:

```
% ./COutput3
Här är en sträng i 3 delar.
%
```

Man kan skriva platshållare helt fast i varandra om så skulle behövas, så ovanstående kunde även skrivas `%s%s%d%s`, förutsatt att man har lämpliga tomrum inne i strängarna istället.

Utskrift av flyttal

Flyttal är en aning speciella att skriva ut i C, precis som i C++. Man kan justera antalet decimaler som talet skall skrivas ut med, samt även den minimala fältbredden som skall användas. För att justera antalet decimaler kan man sätta en punkt (.) före `%i` `%f` och efter punkten skriva antalet decimaler som önskas. T.ex.:

```
#include <stdio.h>
#include <math.h>

int main () {
```

```
printf ("3 decimaler: %.3f\n", M_PI );
printf ("6 decimaler: %.6f\n", M_PI );
printf ("0 decimaler: %.0f\n", M_PI );
}
```

Vi har här inkluderat `<math.h>` för att få tillgång till en definition av π med hög precision. Vi skriver sedan ut π med olika antal decimaler. Programmet ger följande utskrift:

```
% ./COutput4
3 decimaler : 3.142
6 decimaler : 3.141593
20 decimaler: 3.14159265358979311600
0 decimaler : 3
%
```

Om ingen precision ges med hjälp av en punkt och ett tal skrivs talet ut med sex decimaler.

Det finns ett stort antal olika sätt som utskrifter kan manipuleras då man använder sig av `printf()`. Mera information finns i man-sidan för `printf()`.

Utskrift teckenvis

Ibland kan det vara bra att kunna skriva ut tecken till skärmen ett i sänder. I så fall kanske det är onödigt att använda sig av något i stil med:

```
printf("%c", Tecken);
```

för att åstadkomma utskriften, eftersom `printf()` måste göra en hel del extra jobb innan tecknet `Tecken` kan skrivas ut. Det finns en funktion som kan användas för att skriva ut ett enda tecken på skärmen, nämligen:

```
#include <stdio.h>
```

```
int putchar(int c);
```

Denna funktion skriver ut tecknet `c` direkt till skärmen. Vi kan skriva ut innehållet i en C-sträng med följande funktion:

```
#include <stdio.h>
```

```
void printString (const char * String) {
    // iterera över alla tecken
    for (unsigned int Index = 0; Index < strlen(String); Index++ ) {
        putchar ( String[Index] );
    }
}
```

```
int main () {
    char * S = "Denna sträng skall skrivas ut.\n";

    // skriv ut strängen
    printString ( S );
}
```

Programmet skriver helt enkelt ut strängen `S` på skärmen.

[Förutgående](#)

Input/output på C:s vis

[Hem](#)

[Upp](#)

[Nästa](#)

Utskrift till filer